COMPILER DESIGN

Compiler-Based Static Code Analysis and Bug Detection System

Review 1 • Proposed Project

B.Tech CSE | VIT Vellore | 3-member team

Compiler concepts → program analysis → early bug detection

Problem Statement

Why detect issues before execution?

THE PROBLEM

Potential issues may remain hidden until execution, testing, or debugging.

- Late discovery increases debugging effort.
- Not every issue requires program execution to be identified.
- Compiler representations can expose useful program information.

EARLY DETECTION

TRADITIONAL FLOW

Compile
↓
Run / Test
↓
Issue discovered

STATIC ANALYSIS

Source Code
↓
Analyze
↓
Issue detected

Project focus: analyze selected issues without executing the program.

Motivation & Need

Why apply compiler techniques to static analysis?

01

Reduce late debugging

Identify selected issues earlier in the development process.

02

Analyze without execution

Use source-level representations to reason about potential issues.

03

Use compiler structures for analysis

AST, symbol tables, CFGs and data-flow information provide analysis context.

04

Go beyond a lab exercise

Integrate multiple compiler phases into one practical analysis system.

Objectives & Scope

Define a realistic C-like subset and a bounded analysis scope.

OBJECTIVES

- Design a C-like language subset suitable for static analysis.
- Build a multi-phase compiler front-end.
- Perform semantic and scope analysis using symbol tables.
- Perform control-flow/data-flow analysis for selected issues.
- Generate a clear human-readable error and warning report.

SCOPE

Language constructs

- Variables
- Basic data types
- Expressions & assignment
- Conditionals & loops
- Basic functions

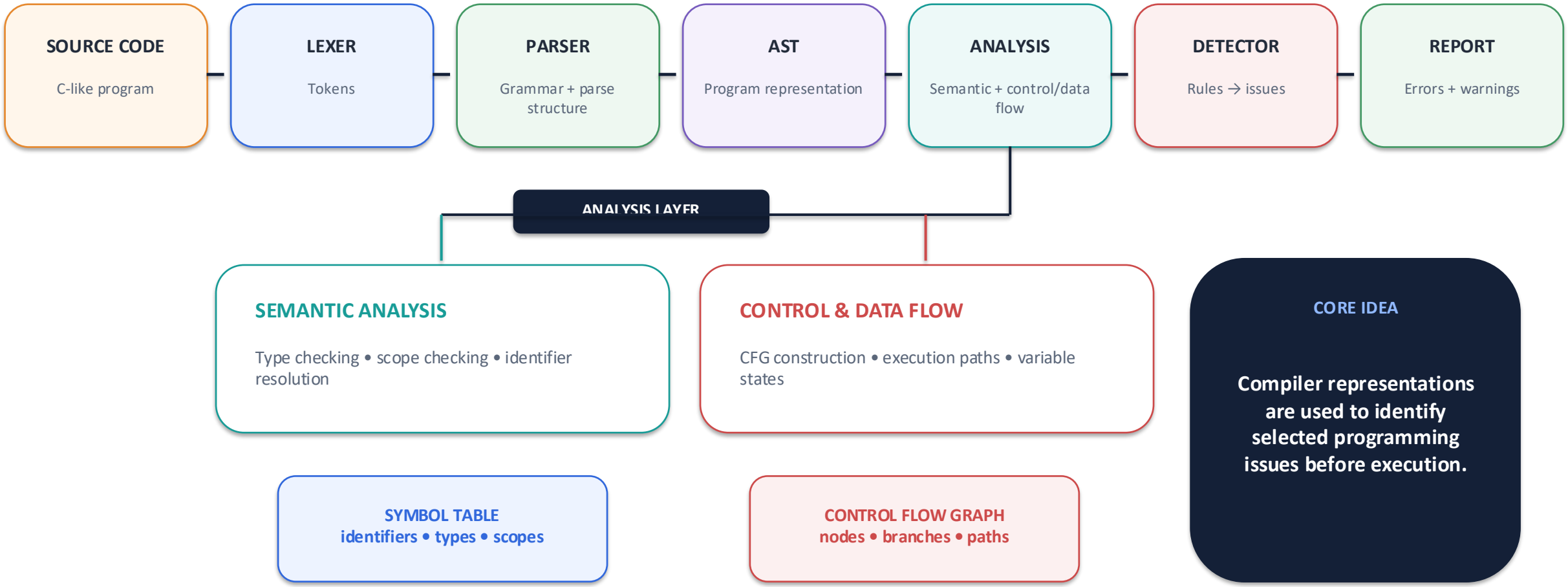
Planned analysis checks

- Undeclared variables
- Type mismatches
- Scope violations
- Uninitialized variables
- Unused variables
- Unreachable code
- Constant division by zero

Defined C-like subset • not the complete C language

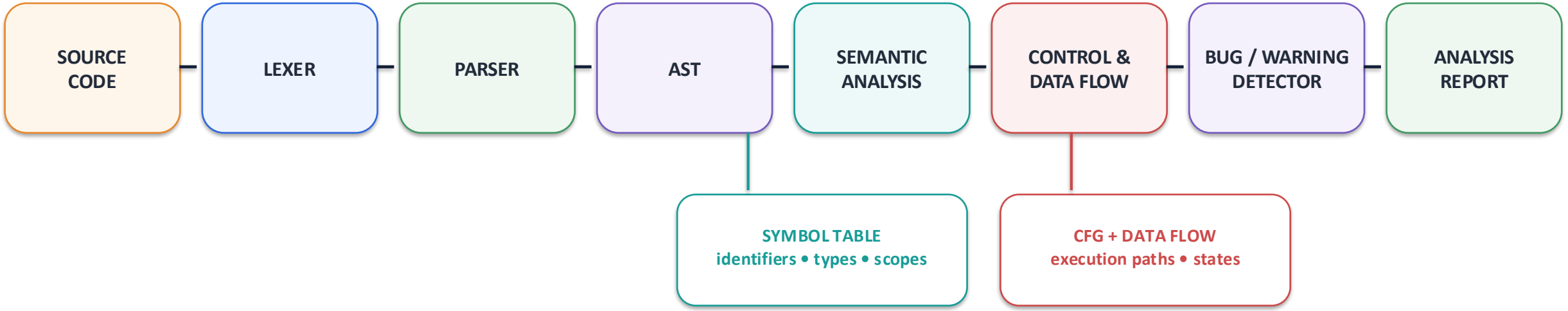
Proposed System Architecture

End-to-end view of the compiler-based analysis pipeline.



Proposed Methodology / Compiler Pipeline

How source code moves through the analysis system.



ANALYSIS OUTPUT — Errors | Warnings | Line numbers | Descriptions

Compiler concepts are integrated rather than implemented as isolated lab exercises.

Compiler Concepts Used

Each compiler concept has a specific role in the analysis pipeline.

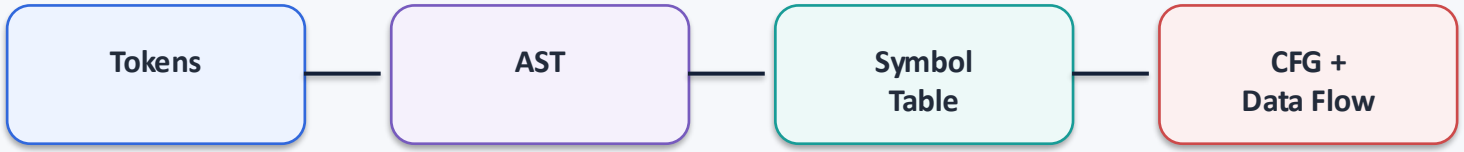
COMPILER CONCEPT	ROLE IN OUR PROJECT
Lexical Analysis	Converts source code into tokens
Syntax Analysis	Validates grammar and builds structure
AST	Represents program structure
Symbol Table	Stores identifiers, types, scopes and attributes
Semantic Analysis	Performs type and scope checks
Control Flow Graph	Represents possible execution paths
Data Flow Analysis	Tracks information such as variable initialization
Static Analysis	Detects potential issues without execution
Error / Warning Reporting	Presents detected issues to the developer

Example: From Source Code to Detection

Illustrative expected behavior for Review 1.

INPUT • C-LIKE CODE

```
int x;
if (x > 10) {
    return 0;
}
y = x + 5;
```



EXPECTED ANALYSIS OUTPUT

WARNING x may be used before initialization.

ERROR y is not declared.

Proposed behavior • final results depend on the implemented rule set.

Proposed Detection Categories

Planned checks within the defined project scope.

SEMANTIC ISSUES

- Undeclared variable
- Duplicate declaration
- Type mismatch
- Scope violation

INITIALIZATION ISSUES

- Use before initialization
- Potentially uninitialized variable

CONTROL-FLOW ISSUES

- Unreachable code
- Simple infinite-loop conditions

CODE QUALITY / SAFETY

- Unused variables
- Constant division by zero

All checks are proposed for the defined C-like subset; they are not claims of full C coverage.

Technology Stack & Team Plan

Keep implementation choices realistic and modular.

TECHNOLOGY OPTIONS

Compiler / Analysis	Implementation language: Python
Lexical analysis	Flex / JFlex, if appropriate
Parsing	Bison / Yacc, if appropriate
CFG visualization	Graphviz
Interface / report	HTML / CSS / JavaScript or lightweight GUI

3-MEMBER RESPONSIBILITY DIVISION

01	**Compiler Front-End**	Lexer • Grammar • Parser • AST
02	**Semantic Analysis**	Symbol table • Type checking • Scope
03	**Static Analysis**	CFG • Data flow • Detection • Reporting

Integration + testing: collaborative

Review Roadmap & Expected Outcome

A staged path from proposal to an integrated analyzer.

01 REVIEW 1

Problem • objectives • scope
Architecture • methodology • compiler concepts

02 REVIEW 2

Front-end • AST • symbol table
Semantic analysis • initial checks

03 REVIEW 3

CFG/data flow • bug detection
Reporting interface • testing

EXPECTED FINAL OUTCOME

A working compiler-based static analysis system for a defined C-like subset, generating meaningful error and warning reports without executing the program.

PROPOSED → IMPLEMENTED